

ChatApp — User Story Document

Version: 1.1

Date: 2026-07-17

1. Authentication

US-01: Email + OTP Registration

As a new user,

I want to register using my email address and verify it with a One-Time Password (OTP),
So that only real users with valid emails can access the app.

Acceptance Criteria:

- User enters name, email, and password on the register screen.
 - System sends a 6-digit OTP to the provided email.
 - User enters OTP on the verification screen within 5 minutes.
 - If OTP is wrong or expired, user can request a new one.
 - After successful OTP verification, account is created and user is logged in automatically.
 - Unverified accounts cannot access any chat feature.
-

US-02: Email Login

As a registered user,

I want to log in using my email and password,
So that I can access my chats securely.

Acceptance Criteria:

- Login form accepts email and password.
 - All inputs are validated using express-validator on the server.
 - JWT token is issued on successful login.
 - A global exception handler returns clean error messages.
 - Failed attempts show a clear error message.
-

2. One-to-One Chat

US-03: User Discovery & Invitation

As a logged-in user,

I want to search for other users and send them an invitation,
So that I can start a private conversation with someone I know.

Acceptance Criteria:

- User can search for other users using their email, phone number, or username.
 - Partial search queries are supported — results appear as the user types.
 - Matching users are displayed in a result list showing name, avatar, and connection status.
 - User can send an invitation directly from the search results.
 - When an invitation is sent, the recipient receives a real-time in-app notification via Socket.IO.
 - The notification includes the sender's information, an Accept option, and a Reject option.
 - The recipient can Accept or Reject the invitation from within the notification.
 - If the recipient accepts, both users are automatically connected and a private chat is created immediately.
 - If the recipient rejects, the sender receives a notification indicating the invitation was rejected.
 - After a rejection, the sender cannot send another invitation to the same user for 24 hours.
 - During the 24-hour cooldown, any attempt to re-invite the same user is blocked with a clear message.
-

US-04: Real-Time Messaging

As a user in a private chat,
I want to send and receive messages instantly,
So that my conversation feels real-time and fluid.

Acceptance Criteria:

- Messages are delivered in real time using Socket.IO.
 - Message status indicators show:
 - ✓ (single grey tick) = Sent (stored on server)
 - ✓✓ (double grey tick) = Delivered (recipient online)
 - ✓✓ (double blue tick) = Read (recipient opened the chat)
 - Status updates happen automatically without a page refresh.
 - Offline messages are stored and delivered when user reconnects.
-

US-05: Infinite Scroll for Chat History

As a user in a chat window,
I want older messages to load automatically as I scroll up,
So that I can read past conversations without clicking.

Acceptance Criteria:

- First 50 messages are loaded when chat opens.
 - Scrolling to the top triggers loading the next 50 messages.
 - A skeleton loading placeholder appears while fetching.
 - Scroll position is preserved after older messages load.
-

US-06: File & Image Attachments (Multer Upload)

As a user in a chat,
I want to send images and files to the other person,
So that we can share media without leaving the app.

Acceptance Criteria:

- Supported file types: JPEG, PNG, GIF, PDF, DOCX, MP4.
 - Files exceeding the size limit are rejected immediately on the client before any upload begins.
 - Unsupported file types show an error message before upload starts.
 - Images are compressed to low quality on the client before uploading to reduce bandwidth.
 - A preview of the selected image or file is shown to the user before they confirm sending.
 - A skeleton / progress indicator is shown in the chat in place of the message while uploading.
 - The file is uploaded to the server via Multer and stored in server storage.
 - Once upload completes, the message appears in the chat with the attachment.
 - Maximum file size: 25 MB per file.
-

US-07: Attachment Edge Cases

As a user uploading a file,
I want the upload to recover gracefully if something goes wrong,
So that I never lose a file or get stuck in an error state.

Acceptance Criteria:

- If internet disconnects mid-upload, a "Reconnecting..." indicator is shown and the upload automatically resumes when connection is restored.
 - If the user closes the browser tab while uploading, the frontend sends a signal to the server before closing.
 - The server marks that message as "upload interrupted" and saves the upload state.
 - If upload fails completely after retries, the user sees a clear error with a "Retry" button.
-

3. Group Chat

US-08: Create a Group

As a user,
I want to create a group chat with multiple contacts,
So that I can communicate with several people at once.

Acceptance Criteria:

- User can create a group with a name and select members from their existing connections.
- A minimum of 2 other members is required.

- The creator is automatically the group admin.
 - Group is created instantly and appears in all members' chat lists.
-

US-09: Group Invitations

As a group admin,
I want to invite new people to the group,
So that the group can grow over time.

Acceptance Criteria:

- Admin can search and send group invitations to any user by email, phone number, or username.
 - Invited user gets a real-time in-app notification with sender information and Accept / Reject options.
 - User can Accept or Decline the group invitation from within the notification.
-

US-10: Group Messaging & Attachments

As a group member,
I want full messaging and attachment features in the group,
So that the group chat is as powerful as a private chat.

Acceptance Criteria:

- All features from one-to-one chat apply:
 - Single / double / blue tick per member.
 - Real-time delivery and read receipts.
 - Infinite scroll for history.
 - File and image attachments with the same Multer upload flow.
 - Typing indicators for all members.
 - Each message shows the sender's name and avatar.
 - Group admin can remove members.
-

4. Chat List & Unread Messages

US-11: Chat List with Unread Badge

As a user,
I want to see all my conversations in a list sorted by most recent activity,
So that I always know which chats need my attention first.

Acceptance Criteria:

- Sidebar shows all private and group conversations.
- Each item shows: avatar, name, last message preview, timestamp, and an unread message count badge.
- The conversation with the newest message floats to the top.
- Last message preview updates in real time when a new message arrives.

- Unread badge disappears when the user opens that chat.
-

US-12: Last Unread Message Jump

As a user opening a chat with unread messages,
I want to jump directly to the first unread message,
So that I don't have to scroll to find where I left off.

Acceptance Criteria:

- When opening a chat with unread messages, the view scrolls to the first unread message automatically.
 - A "Jump to unread" button appears if the user scrolls away from the unread position.
 - After reading, unread count clears and ticks update to blue.
-

5. Performance & Caching

US-13: In-Memory Caching with Redis

As a developer,
I want frequently accessed data cached in Redis,
So that the server handles high traffic without hitting the database on every request.

Acceptance Criteria:

- Cache covers: user profile data, conversation lists, recent messages (last 50), online user status.
 - TTL (Time to Live): 5 minutes for messages, 1 hour for user profiles.
 - On new message, the relevant cache entry is invalidated and refreshed.
 - Redis is used as the primary cache; MemCache as the fallback layer.
 - On a cache miss, data is fetched from MongoDB and the cache is populated.
-

US-14: Database Redundancy

As a system operator,
I want the database to be redundant and fault-tolerant,
So that no messages are lost if a server goes down.

Acceptance Criteria:

- MongoDB replica set (Primary + 2 Secondaries).
 - All writes are confirmed on at least 2 nodes before acknowledging success.
 - Automatic failover occurs if the primary node goes down.
 - Messages are stored permanently; no data is deleted without explicit user action.
-

6. Notifications

US-15: Real-Time In-App Notifications

As a user,

I want to receive notifications inside the app when I get a new message or invitation,
So that I don't miss anything even if I'm not in that chat.

Acceptance Criteria:

- New messages trigger a notification badge on the chat icon.
 - Invitations show a badge on the notification bell.
 - All notifications are delivered via Socket.IO in real time.
 - Clicking a notification navigates the user directly to that chat.
-

US-16: Background Push Notifications (Worker)

As a user who is not actively using the app,

I want to still receive a push notification when I get a new message,
So that I know to come back to the app.

Acceptance Criteria:

- A background worker (e.g., Bull Queue + Node worker) listens for new message events when the recipient is offline.
 - Worker sends an email or push notification to the offline user.
 - Worker retries failed notification delivery up to 3 times.
 - Background worker runs independently from the main API server.
-

7. Technical & Architecture Requirements

US-17: Backend Architecture (MVC Pattern)

As a developer,

I want the backend to follow a strict MVC architecture,
So that the codebase is easy to maintain and extend.

Acceptance Criteria:

- Models: Mongoose schemas (User, Message, Conversation, Invitation, Notification, etc).
 - Controllers: One controller per feature (authController, chatController, etc.).
 - Routes: Express routers map endpoints to controllers.
 - Services: Business logic lives in a /services layer, not inside controllers.
 - Views: Not applicable (API-only backend).
-

US-18: Input Validation & Error Handling

As a developer,

I want every API endpoint to validate input and return structured errors,
So that the app is secure and easy to debug.

Acceptance Criteria:

- express-validator validates all request bodies and params.
 - A single global middleware catches all unhandled errors.
 - A global exception handler catches unhandled Promise rejections and process-level errors.
 - All errors returned as: { success: false, message: "...", errors: [...] }
 - HTTP status codes used correctly (400, 401, 403, 404, 500).
-

US-19: Handling Multiple Concurrent Users

As a user of the platform,

I want the app to work smoothly even when thousands of users are chatting at the same time,

So that I never experience slowness or missed messages.

Acceptance Criteria:

- Socket.IO uses Redis Adapter for multi-server pub/sub.
 - Express uses connection pooling for MongoDB.
 - Rate limiting applied to API routes to prevent abuse.
 - Load balancer ready architecture (stateless JWT auth).
 - Horizontal scaling supported via Docker/PM2.
-

US-20: Frontend Architecture (React In-Place)

As a developer,

I want the React frontend to be component-based and handle all edge cases gracefully,

So that users never see broken UI or silent failures.

Acceptance Criteria:

- Each feature is a self-contained React component.
 - Skeleton loaders shown during any data fetch.
 - All async operations handle loading, success, and error states.
 - Optimistic UI updates for message sending (message shows immediately, reverts if server fails).
 - Network offline detection shows a banner ("You are offline").
 - Error boundaries catch unexpected component crashes.
-